

Faculdade de Engenharia da Universidade do Porto



Base de Dados

Criação de uma base de dados para uma imobiliária

Projeto de Base de Dados - Licenciatura em Engenharia Informática e Computação

Regentes da cadeira: Carla Alexandra Teixeira Lopes e Michel Celestino Paiva Ferreira

Professor da cadeira: Lázaro Gabriel Barros da Costa

Grupo:

Afonso Tavares Fernandes de Almeida Saraiva - up202304461

Sara Marques Ribeiro - up202305327

Index

Task 4	3
Context description	3
UML	5
Initial Concept Model	5
Generative AI Integration	5
Task 5	7
Relational Schema	7
Initial Relational Schema	7
Generative AI Integration	8
Final Relational Schema	8
Functional Dependencies and Normal Forms Analysis	9
Initial proposal	9
Generative AI assistance	13
Sequence of Prompts:	13
Outcome:	13
Conclusion:	14
Final proposal	14
SQLite Database Creation	18
Data Loading	18
Generative AI Integration	19
Contribution	20

Task 4

Context description

The real estate agency operates across various neighborhoods within a city and requires a system to efficiently manage its core operations. Its database needs to vary.

The agency's operating neighborhoods must have a name and a neighborhoodID. Each property can only be linked to one neighborhood, but the neighborhoods can have several properties linked to them and the location of both.

Geographical data, such as latitude, and longitude, are held by a specific Location object that is dependent on properties. This object facilitates the accurate determination of each property's exact position.

The system holds information on properties, including propertyID, name, address, type (connected to PropertyType), and value (representing sale or rental values), since properties are the main assets of the real estate company. Additionally, a property can have multiple documents associated with it. However, each property is managed by only one agency.

The PropertyType entity defines the classification of properties (house, apartment, land). It holds a propertyTypeID and a description to clarify the type of property.

Agencies manage the properties and have an agencyID, name, and address. An agency can handle multiple properties and is responsible for coordinating the activities of its real estate agents.

Agents are employees of the agency who handle transactions. Each agent has an agentID and a transactionsNumber to track the number of transactions they have handled. The relationship between agents and transactions is managed through the AgentTransaction entity, which includes details like the agentRole and the assignedDate for each transaction they handle.

Person stores personal information for agents and clients, with attributes such as name, phoneNumber, and email.

Clients are individuals or organizations that interact with the agency for buying, selling, or renting properties. Each client has a clientID, address, and a type (buyer or seller). Clients can participate in multiple transactions over time and are connected to specific transactions and appointments.

Transactions representing the buying, selling, or renting of properties. Each transaction includes a unique identifier (transactionID), the date it occurred, value,

and type (buy, sell, rent). Transactions can involve multiple properties and are linked to a single client. They may also have a corresponding contract.

The TransactionHistory records the changes in status for each transaction, including details like transactionHistoryID, status, and statusDate, to monitor the progression of transactions.

Contracts specify the details of a transaction, including conditions, deadlines, and agreed values. Each contract is stored with a contractID and is linked to a single transaction. However, not every transaction has a contract.

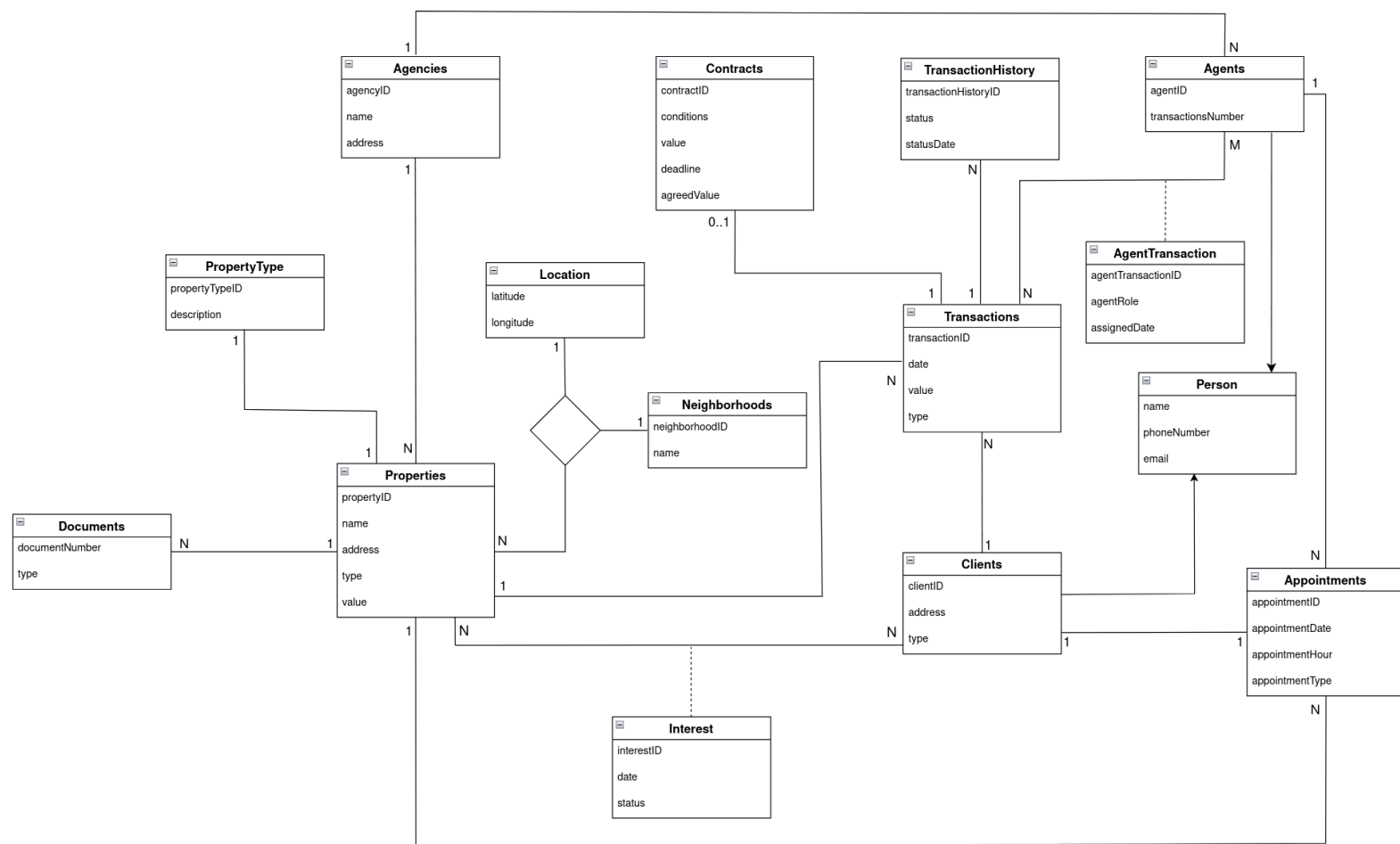
Documents are crucial for formalizing transactions and properties. Each document has a documentNumber, type (purchase agreement, lease, or deed), and can be linked to multiple properties.

To organize property visits and client interactions, the system manages appointments. Each appointment is identified by a unique ID and includes details such as the date, time, and type of appointment specifying the purpose of the visit.

The Interest entity keeps a record of clients' interest in specific properties. It includes fields like InterestID, date, and status.

UML

Initial Concept Model



Generative AI Integration

We began by asking the AI to propose improvements to the initial concept model. It suggested:

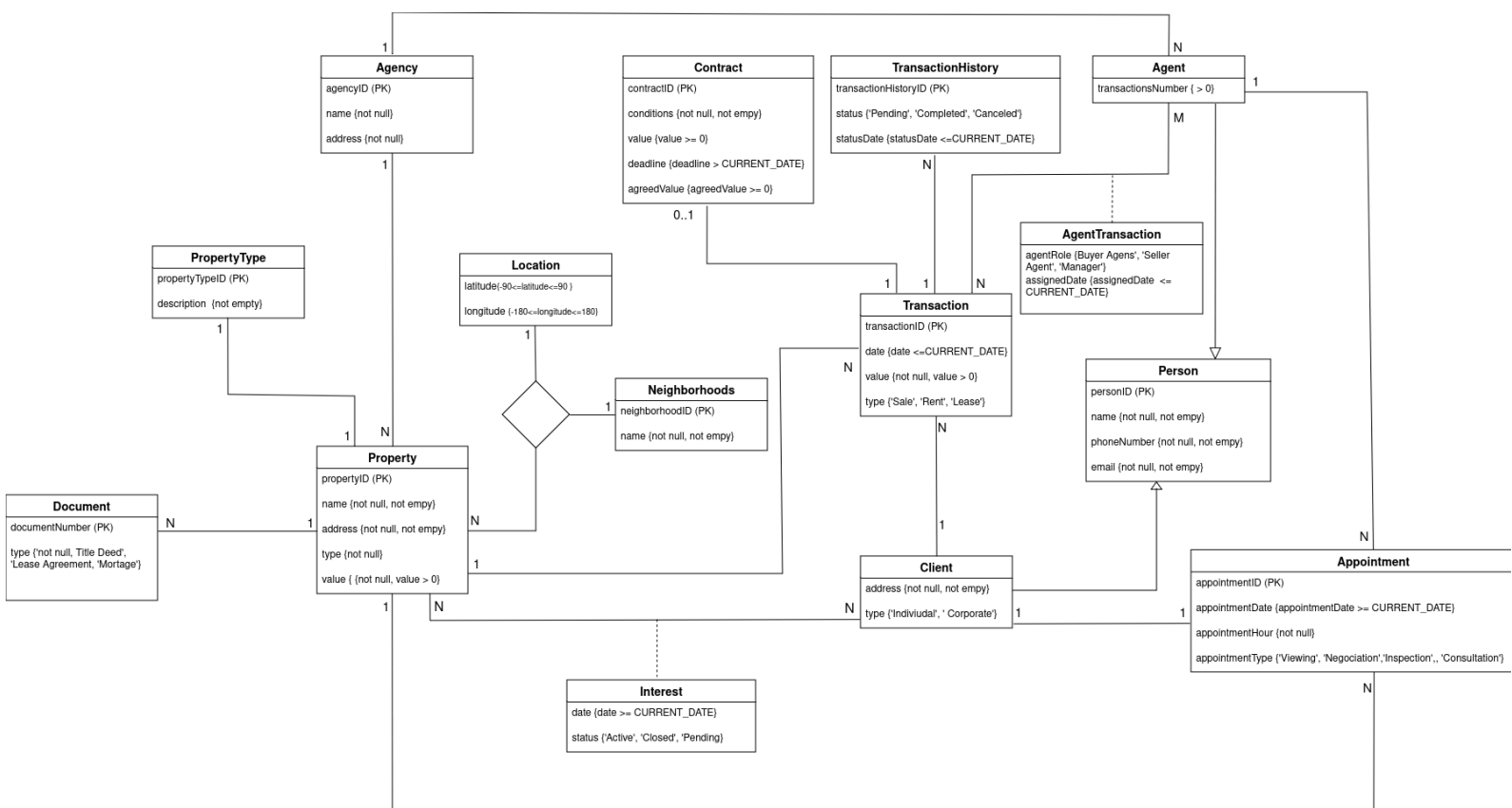
1. **Class Renaming**: To ensure class names better reflect their purpose within the domain.
2. **Adding Constraints**: To formalize the model and ensure data integrity by defining unique identifiers and rules.
3. **Attribute Removal**: Eliminating ClientID, InterestID, AgentID, and AgentTransactionID to simplify and clarify the structure.
4. **Inclusion of PersonID**: To accurately represent unique individuals within the system.

5. Arrow Adjustments: Refining associations and dependencies for better clarity and more accurate relationship representation.

ChatGPT also emphasized reducing clutter by organizing relationships and leveraging aggregation or composition to depict stronger associations. A particularly notable recommendation was to introduce a TransactionHistory class. This provided a cleaner way to handle multiple transaction records, addressing the complexity of one-to-many relationships and adding depth to the model.

The detailed suggestions helped align the UML diagram with best practices, improving its clarity, readability, and adherence to the task requirements

Final Concept Model



Task 5

Relational Schema

Initial Relational Schema

Agency (agencyID, name, address)

Property (propertyID, name, address, type, value, agencyID -> Agency, propertyTypeID->PropertyType, latitude->Location, longitude->Location, neighborhoodID->Neighborhood)

PropertyType (propertyTypeID, description)

Document (documentNumber, type, propertyID->Property)

Location (latitude, longitude)

Neighborhood (neighborhoodID, name)

Appointment (appointmentID, appointmentDate, appointmentHour, appointmentType, propertyID->Property, agentID->Agent)

Agent (agentID->Person, transactionsNumber, agencyID->Agency)

Transaction (transactionID, date, value, type, propertyID->Property, contractID->Contract, clientID->Client)

TransactionHistory (transactionHistoryID, status, statusDate, transactionID->Transaction)

Contract (contractID, conditions, value, deadline, agreedValue)

Client (clientID->Person, address, type)

Interest (date, status, propertyID->Property, clientID->Client)

AgentTransaction (agentRole, assignedDate, agentID->Agent, transactionID->Transaction)

Person (personID, name, phoneNumber, email)

PropertiesLocationNeighborhoods(latitude->Location, longitude->Location, neighborhoodID->Neighborhood, propertyID->Property)

Generative AI Integration

To refine the schema, we interacted with ChatGPT to ensure that the relational model aligned with best practices while retaining the original UML design. Specifically, one key interaction was about the handling of the PropertiesLocationNeighborhoods table, this was the only part about the schema that required adjustment.

ChatGPT suggested removing the PropertiesLocationNeighborhoods table since it was redundant. It recommended integrating latitude, longitude, and neighborhoodID directly into the Property table. After considering this suggestion, the PropertiesLocationNeighborhoods table was removed, and its attributes were added directly to the Property table. The revised Property relation now looks like this

The change simplified the schema by reducing unnecessary complexity, aligning it more closely with the conceptual model, and improving query efficiency without altering the relationships presented in the UML

Final Relational Schema

Agency (agencyID, name, address)

Property (propertyID, name, address, type, value, agencyID -> Agency, propertyTypeID->PropertyType, latitude->Location, longitude->Location, neighborhoodID->Neighborhood)

PropertyType (propertyTypeID, description)

Document (documentNumber, type, propertyID->Property)

Location (latitude, longitude)

Neighborhood (neighborhoodID, name)

Appointment (appointmentID, appointmentDate, appointmentHour, appointmentType, propertyID->Property, agentID->Agent)

Agent (agentID->Person, transactionsNumber, agencyID->Agency)

Transaction (transactionID, date, value, type, propertyID->Property, contractID->Contract, clientID->Client)

TransactionHistory (transactionHistoryID, status, statusDate,
transactionID->Transaction)

Contract (contractID, conditions, value, deadline, agreedValue)

Client (clientID->Person, address, type)

Interest (date, status, propertyID->Property, clientID->Client)

AgentTransaction (agentRole, assignedDate, personID->Agent,
transactionID->Transaction)

Person (personID, name, phoneNumber, email)

Functional Dependencies and Normal Forms Analysis

Initial proposal

Agency:

FDs:

agencyID -> name, address

Forms:

BCNF: Yes

3NF: Yes

Property:

FDs:

propertyID -> name, address, type, value, agencyID, propertyTypeID,
latitude, longitude, neighborhoodID

Forms:

BCNF: Yes

3NF: Yes

PropertyType:

FDs:

propertyTypeID -> description

Forms:

BCNF: Yes

3NF: Yes

Document:

FDs:

documentNumber -> type, propertyID

Forms:

BCNF: Yes

3NF: Yes

Location:

FDs: ----

Forms:

BCNF: Yes

3NF: Yes

Neighborhood:

FDs:

neighborhoodID -> name

Forms:

BCNF: Yes

3NF: Yes

Appointment:

FDs:

appointmentID -> appointmentDate, appointmentHour,
appointmentType

Forms:

BCNF: Yes

3NF: Yes

Agent:

FDs:

agentID -> transactionsNumber, agencyID

Forms:

BCNF: Yes

3NF: Yes

Transaction:

FDs:

transactionID -> date, value, type, propertyID, contractID, clientID

Forms:

BCNF: Yes

3NF: Yes

TransactionHistory:

FDs:

transactionHistoryID -> status, statusDate, transactionID

Forms:

BCNF: Yes

3NF: Yes

Contract:

FDs:

contractID -> conditions, value, deadline, agreedValue

Forms:

BCNF: Yes

3NF: Yes

Client:

FDs:

clientID -> address, type

Forms:

BCNF: Yes

3NF: Yes

Interest

FDs:

propertyID, clientID -> date, status

Forms:

BCNF: Yes

3NF: Yes

AgentTransaction

FDs:

agentID, transactionID -> agentRole, assignedDate

Forms:

BCNF: Yes

3NF: Yes

Person

FDs:

personID-> name, phoneNumber, email

Forms:

BCNF: Yes

3NF: Yes

In all relations where there are dependencies, the left side of the dependency is a superkey and determines all other attributes, thus satisfying the BCNF condition. Each relation satisfies the 3NF condition because there are no transitive dependencies, and every non-prime attribute (any attribute not part of the candidate key) is fully dependent on the candidate key (which ensures 2NF).

Generative AI assistance

In this analysis, we utilized OpenAI's **ChatGPT**, a generative AI tool, to review the functional dependencies and normal form compliance of a relational database schema. The process involved uploading the schema and providing detailed prompts to guide the AI in analyzing the schema's adherence to BCNF and 3NF normalization principles.

Sequence of Prompts:

Initial Prompt: We began by sharing the relational schema, including its functional dependencies and a detailed analysis of normal forms, and asked for recommendations to improve the design.

Follow-Up Prompt: After providing additional clarification and confirming the schema's compliance with BCNF and 3NF, we sought further suggestions or insights to optimize the schema for practical implementation.

Outcome:

Unfortunately, ChatGPT did not provide any new or useful tips beyond confirming that the schema adheres to BCNF and 3NF. While the tool reiterated the schema's compliance and provided a review, it failed to offer practical suggestions or identify areas for improvement that had not already been covered in the initial analysis.

Conclusion:

While generative AI tools like ChatGPT can confirm existing work, their capacity to provide deeper insights or novel suggestions is limited when the provided input is already comprehensive and well-structured. This reinforces the importance of pairing AI assistance with human expertise for nuanced database design.

Final proposal

The final proposal for the database schema remains the same as the initial design. After analyzing the schema and its functional dependencies using OpenAI's ChatGPT, it was confirmed that the functional dependencies are correct and the schema is already optimized according to the principles of BCNF and 3NF.

Agency:

FDs:

agencyID -> name, address

Forms:

BCNF: Yes

3NF: Yes

Property:

FDs:

propertyID -> name, address, type, value, agencyID, propertyTypeID,
latitude, longitude, neighborhoodID

Forms:

BCNF: Yes

3NF: Yes

PropertyType:

FDs:

propertyTypeID -> description

Forms:

BCNF: Yes

3NF: Yes

Document:

FDs:

documentNumber -> type, propertyID

Forms:

BCNF: Yes

3NF: Yes

Location:

FDs: ----

Forms:

BCNF: Yes

3NF: Yes

Neighborhood:

FDs:

neighborhoodID -> name

Forms:

BCNF: Yes

3NF: Yes

Appointment:

FDs:

appointmentID -> appointmentDate, appointmentHour,
appointmentType

Forms:

BCNF: Yes

3NF: Yes

Agent:

FDs:

agentID -> transactionsNumber, agencyID

Forms:

BCNF: Yes

3NF: Yes

Transaction:

FDs:

transactionID -> date, value, type, propertyID, contractID, clientID

Forms:

BCNF: Yes

3NF: Yes

TransactionHistory:

FDs:

transactionHistoryID -> status, statusDate, transactionID

Forms:

BCNF: Yes

3NF: Yes

Contract:

FDs:

contractID -> conditions, value, deadline, agreedValue

Forms:

BCNF: Yes

3NF: Yes

Client:

FDs:

clientID -> address, type

Forms:

BCNF: Yes

3NF: Yes

Interest

FDs:

propertyID, clientID -> date, status

Forms:

BCNF: Yes

3NF: Yes

AgentTransaction

FDs:

agentID, transactionID -> agentRole, assignedDate

Forms:

BCNF: Yes

3NF: Yes

Person

FDs:

personID-> name, phoneNumber, email

Forms:

BCNF: Yes

3NF: Yes

In all relations where there are dependencies, the left side of the dependency is a superkey and determines all other attributes, thus satisfying the BCNF condition.

Each relation satisfies the 3NF condition because there are no transitive dependencies, and every non-prime attribute (any attribute not part of the candidate key) is fully dependent on the candidate key (which ensures 2NF).

SQLite Database Creation

Tool Used:

ChatGPT was used to assist in creating an SQLite database. Its ability to generate SQL code based on prompts made it a helpful tool for this task.

Sequence of Prompts Given:

I started by providing examples from class materials, such as how to create tables with primary keys, foreign keys, and cascading behaviors like ON DELETE CASCADE.

Example Prompt:

"Create an SQLite table for a Property entity, including a primary key and a foreign key with CASCADE."

Project-Specific Tables and Attributes:

I shared the project's UML diagram, listing all tables, attributes, and their relationships.

Example Prompt:

"Using the following UML diagram, create SQLite tables:

Clarifications and Refinements:

When errors or ambiguities were identified, I asked ChatGPT to adjust specific constraints or relationships.

Example Prompt:

"In the Order table, ensure the user_id foreign key references user_id in the User table and include ON DELETE CASCADE."

Refinements Made After AI Integration:

The AI-generated SQL scripts were effective but required minor corrections. A few foreign key relationships were adjusted to reference the correct columns. I also verified and tweaked cascade rules and constraints to match the project's logic.

Data Loading

Tool Used:

ChatGPT was used to assist with generating SQL scripts for data insertion into the

SQLite database. By providing clear inputs, the AI efficiently created insert statements aligned with the project's schema and requirements.

Sequence of Prompts Given:

Providing Table Structure and Attributes:

I began by sharing the project tables, their attributes, and any constraints (e.g., unique fields). This helped ChatGPT understand the structure of the database.

Example Insert Statement:

Then I asked ChatGPT to generate an example insert based on the provided table structure.

Generating Bulk Inserts:

After confirming everything the previous step was correct I asked ChatGPT to create multiple INSERT statements for batch loading data into all the tables. The generated scripts included realistic and varied data to populate the database.

Adding INSERT OR IGNORE for Unique Attributes:

To handle scenarios where the script might be executed multiple times without causing errors due to unique constraints, I asked ChatGPT to modify the statements using INSERT OR IGNORE.

Refinements Made After AI Integration:

The AI-generated scripts were effective, but I made some manual adjustments to fine-tune the data consistency. For instance:

- Ensured foreign key references in child tables matched existing data in parent tables.
- Corrected minor formatting issues in date and numeric fields.
- Verified data types for each column to match the schema.

The final scripts included multiple INSERT statements for bulk data loading, and INSERT OR IGNORE was used after AI recommendation to prevent errors when running the scripts multiple times. These refinements ensured smooth data loading and avoided issues with unique constraints.

Generative AI Integration

Throughout the project, the generative AI tool, ChatGPT, proved to be a valuable asset, though its effectiveness varied across different stages. Its strengths included quickly generating structured outputs, adapting to prompts, and handling repetitive or monotonous tasks. However, limitations arose in complex logic or when a deeper understanding of project-specific nuances was required.

Most Useful Stages:

Improving the Report:

The AI was most useful in drafting and refining the project report. It generated well-structured content, organized ideas cohesively, and saved significant time by transforming raw project details into professional, polished text. Its ability to rephrase and clarify explanations enhanced the overall readability and quality of the report. Additionally, it provided consistency in tone and format, ensuring a seamless presentation of the project.

Data Loading:

The AI significantly simplified the tedious task of creating bulk INSERT statements. By automating the generation of realistic and varied sample data for all tables, it reduced the effort required to manually populate the database. Using prompts, it also added INSERT OR IGNORE functionality, enabling repeated script executions without errors due to unique constraints. This automation was particularly valuable for handling the repetitive nature of data loading.

Less Useful Stages:

Schema and Dependencies:

While the AI provided a solid starting point for creating the database schema, its usefulness was limited in this stage. The AI-generated scripts sometimes required manual adjustments to ensure correct foreign key relationships, cascading rules, and alignment with the project's specific requirements. Its output was more generic and needed close review to identify and fix errors in complex relationships or constraints.

Conclusion:

The AI tool excelled in tasks requiring speed, consistency, and handling repetitive work, such as writing the report and data loading. However, for tasks demanding precise logical dependencies and detailed project-specific adjustments, such as schema design, its limitations became evident. Overall, its integration saved time and effort.

Contribution

We divided the work equally and worked closely together, making the most of our two-person team for a project designed for three. Despite the challenge, we believe we accomplished quality work and learned a lot in the process. It was a rewarding experience, and we enjoyed collaborating and tackling the tasks together.